

LAB PRACTICUM: OpenMP Multi-Core Scaling in Python

Hands-Off Benchmark Practicum • 60 Minutes • Autonomous Execution

Student Name:	Kabdrash Yerassyl	Student ID:	230103041
Hardware / CPU:	Intel64 Family 6 Model 140 Stepping 1, GenuineIntel	Date & Section:	02.10.2026

LAB PROTOCOL & AUTONOMOUS RULES OF ENGAGEMENT

1. Autonomous Session: You are expected to read, run, optimize, and document all findings independently.
2. Zero Support for Dependency Issues: If you encounter an error, verify that you installed packages inside a clean virtual environment. Compilers are NOT needed for this lab.
3. Equal Hardware Standard: Keep your laptop plugged into wall power and close background browsers, video streams, or games to prevent CPU thermal throttling and frequency capping.
4. Whiteboard Leaderboard: When you finish Task 1 and Task 3, record your CPU model and best speedup factor on the lab whiteboard.

1. Background & Environment Setup (10 Minutes)

Standard CPython executes bytecode under the Global Interpreter Lock (GIL), preventing multiple CPU threads from executing simultaneously in Python space. In this lab, we bypass the GIL using Numba's JIT compiler with its native multithreaded OpenMP/TBB work-stealing engine. Because Numba compiles numeric Python loops directly to machine code and releases the GIL inside parallel loops (prange), we achieve bare-metal native CPU scaling across both macOS (Apple Silicon M-series) and Windows/Linux x86_64 machines with zero local C++ compiler dependencies.

Execute the following in your terminal / command prompt:

```
pip install numpy numba matplotlib
```

2. Challenge 1: The Core Speedometer & Amdahl's Law

Objective: Benchmark an embarrassingly parallel Monte Carlo simulation to approximate π across varying thread counts. You will measure execution latency, quantify parallel speedup, and observe where Amdahl's Law caps performance.

```
import time
import numpy as np
import numba
from numba import njit, prange

print(f"Hardware Threads Detected: {numba.config.NUMBA_NUM_THREADS}")

@njit(parallel=True)
def monte_carlo_pi(n_samples):
    inside_circle = 0
    for i in prange(n_samples):
        x = np.random.uniform(0.0, 1.0)
        y = np.random.uniform(0.0, 1.0)
        if x * x + y * y <= 1.0:
            inside_circle += 1
    return (4.0 * inside_circle) / n_samples

# JIT Warmup (excludes JIT compilation overhead from benchmarks)
_ = monte_carlo_pi(10_000)

SAMPLES = 120_000_000
thread_counts = [1, 2, 4, 8, numba.config.NUMBA_NUM_THREADS]
# Filter out duplicates and invalid counts
thread_counts = sorted(list(set([t for t in thread_counts if t <=
numba.config.NUMBA_NUM_THREADS])))

print(f"{'Threads':<10} | {'Time (s)':<12} | {'Speedup':<10} | {'Efficiency (%)':<15}")
print("-" * 55)

t1_baseline = None
for t in thread_counts:
    numba.set_num_threads(t)
    start = time.perf_counter()
    pi_est = monte_carlo_pi(SAMPLES)
    elapsed = time.perf_counter() - start

    if t == 1:
        t1_baseline = elapsed
        speedup = 1.0
        efficiency = 100.0
    else:
```

```

speedup = t1_baseline / elapsed
efficiency = (speedup / t) * 100.0

print(f"{t:<10} | {elapsed:<12.4f} | {speedup:<10.2f}x |
{efficiency:<15.1f}")

```

Deliverables & Technical Questions for Challenge 1:

- A. Record your baseline T_1 (1 thread) and your maximum multi-thread runtime $T_{\max}$ in Table 1 below.
- B. Why is the warm-up call `_ = monte_carlo_pi(10_000)` strictly required before recording benchmark timings?
- C. Did your parallel efficiency stay at 100% when scaling from 1 to max threads? What architectural factors (e.g. memory bus contention, hyperthreading/SMT core vs physical core limits) explain the degradation?
- D. Why does `inside_circle += 1` inside `prange` not corrupt the count? What implicit OpenMP reduction construct does Numba generate here?

A. $T_1 = 2.5504$ s and $T_{\max}$ (8 threads) = 0.5793 s, a speedup of 4.40x. B. Numba compiles the function the first time it is called. Without the warm-up, the first timed run would include JIT compilation and thread-pool startup, not just the computation. T_1 would be inflated and the speedups would look better than they really are. The warm-up call with 10,000 samples triggers compilation before timing starts. C. No. Efficiency fell from 100% to 90.7% (2 threads), 72.7% (4 threads) and 55.0% (8 threads). Likely causes: Hyperthreading (SMT): the 8 threads are probably logical threads on fewer physical cores. Two threads on one core share its execution units, so each extra thread adds much less than a full core. Clock frequency: with all cores busy, power and thermal limits lower the clock. A single thread can run at a higher turbo frequency. Amdahl's Law: the serial parts (thread startup, the final reduction, the Python call) don't speed up. From the 8-thread result, the effective non-scaling fraction is about 12%, which would cap speedup at roughly $1/0.12 \approx 8x$. This estimate also absorbs the hardware effects above.

Memory bus contention is small here, because Monte Carlo barely touches memory.

D. Numba recognizes `inside_circle += 1` inside `prange` as a reduction. Each thread gets its own private copy of `inside_circle`, starting at 0, and increments only that copy. After the loop, the private copies are summed into the final value. No two threads write to the same variable, so there is no race condition. This is the equivalent of OpenMP's `reduction(+:inside_circle)`.

3. Challenge 2: Load Imbalance & Dynamic Scheduling

Objective: Compute a high-resolution 2500×2500 Mandelbrot fractal. Unlike Monte Carlo, points in the complex plane require radically different amounts of compute: points inside the set iterate all the way to `max_iter`, while outer points escape in 2-3 iterations. You will investigate how non-uniform workloads impact thread utilization.

```
import time
import numpy as np
import matplotlib.pyplot as plt
from numba import njit, prange

@njit(parallel=True)
def render_mandelbrot_rows(h, w, max_iter):
    img = np.zeros((h, w), dtype=np.int32)
    # Row-parallel loop
    for r in prange(h):
        cy = -1.2 + (r / h) * 2.4
        for c in range(w):
            cx = -2.0 + (c / w) * 2.5
            z_real, z_imag = 0.0, 0.0
            it = 0
            while (z_real * z_real + z_imag * z_imag <= 4.0) and (it <
max_iter):
                next_real = z_real * z_real - z_imag * z_imag + cx
                z_imag = 2.0 * z_real * z_imag + cy
                z_real = next_real
                it += 1
            img[r, c] = it
    return img

@njit(parallel=True)
def render_mandelbrot_cols(h, w, max_iter):
    img = np.zeros((h, w), dtype=np.int32)
    # Column-parallel loop (swapped iteration axis)
    for c in prange(w):
        cx = -2.0 + (c / w) * 2.5
        for r in range(h):
            cy = -1.2 + (r / h) * 2.4
            z_real, z_imag = 0.0, 0.0
            it = 0
            while (z_real * z_real + z_imag * z_imag <= 4.0) and (it <
max_iter):
                next_real = z_real * z_real - z_imag * z_imag + cx
                z_imag = 2.0 * z_real * z_imag + cy
                z_real = next_real
                it += 1
```

```

        img[r, c] = it
    return img

# Warmup JIT
_ = render_mandelbrot_rows(100, 100, 50)
_ = render_mandelbrot_cols(100, 100, 50)

H, W, MAX_IT = 2500, 2500, 1000

t0 = time.perf_counter()
grid_rows = render_mandelbrot_rows(H, W, MAX_IT)
t_rows = time.perf_counter() - t0

t1 = time.perf_counter()
grid_cols = render_mandelbrot_cols(H, W, MAX_IT)
t_cols = time.perf_counter() - t1

print(f"Row-Parallel Render Time:    {t_rows:.3f} s")
print(f"Column-Parallel Render Time: {t_cols:.3f} s")

# Save visual artifact
plt.figure(figsize=(8, 8))
plt.imshow(grid_rows, cmap='magma', extent=[-2.0, 0.5, -1.2, 1.2])
plt.title(f"Mandelbrot {H}x{W} (Render: {t_rows:.2f}s)")
plt.axis('off')
plt.savefig('mandelbrot_output.png', dpi=300, bbox_inches='tight')
print("Saved image: mandelbrot_output.png")

```

Deliverables & Technical Questions for Challenge 2:

- A. Compare `t_rows` vs `t_cols`. Which axis decomposition completed faster on your machine and why?
- B. Explain how NumPy's C-contiguous (row-major) memory layout interacts with CPU cache lines when threads write to `img[r, c]` vs `img[c, r]`.
- C. Load imbalance analysis: If thread 0 receives the top 20% of rows and thread 2 receives the center 20% of rows, why does one thread idle while the other struggles? How does OpenMP dynamic scheduling resolve this?
- D. Attach your exported `mandelbrot_output.png` file alongside your completed task sheet.

4. Challenge 3: Stencil Computation & Memory Bandwidth

Objective: Simulate 2D heat diffusion using a 5-point discrete Laplacian stencil over 300 timesteps on a 1500×1500 grid. Unlike the compute-bound fractal, stencil computations are strictly memory-bandwidth bound. You will determine whether adding CPU cores continues to speed up performance or hits the RAM memory bus wall.

```
import time
import numpy as np
from numba import njit, prange

@njit(parallel=True)
def heat_step(u, u_next, alpha=0.20):
    rows, cols = u.shape
    for i in prange(1, rows - 1):
        for j in range(1, cols - 1):
            u_next[i, j] = u[i, j] + alpha * (
                u[i+1, j] + u[i-1, j] + u[i, j+1] + u[i, j-1] - 4.0 * u[i, j]
            )

GRID_SIZE = 1500
STEPS = 300

u = np.zeros((GRID_SIZE, GRID_SIZE), dtype=np.float64)
u_next = np.zeros_like(u)

# Dirichlet Boundary conditions: Top & Left walls maintained at 100°C
u[0, :] = 100.0
u[:, 0] = 100.0
u_next[0, :] = 100.0
u_next[:, 0] = 100.0

# Warmup
heat_step(u, u_next)

# Benchmark Execution
start = time.perf_counter()
for step in range(STEPS):
    heat_step(u, u_next)
    # Pointer swap (avoids array allocation copies)
    u, u_next = u_next, u
elapsed = time.perf_counter() - start

cells_per_sec = (GRID_SIZE * GRID_SIZE * STEPS) / elapsed / 1e6
print(f"Heat Diffusion Complete: {elapsed:.3f} s")
print(f"Throughput: {cells_per_sec:.2f} Megacells/sec")
```

Deliverables & Technical Questions for Challenge 3:

- A. Record your benchmark runtime and Megacells/sec throughput in Table 1.
- B. Change the array precision from `np.float64` to `np.float32`. Re-run the test. By what factor did runtime decrease? Why does halving memory footprint improve stencil throughput far more than compute capability?
- C. Memory Roofline Model: Explain why adding twice as many CPU cores does not produce a 2x speedup on stencil codes once memory bandwidth is saturated.

A. The float64 run took 1.397 s, which is 483.04 Megacells/sec. B. With float32, runtime fell from 1.397 s to 0.647 s, a factor of about 2.16x (throughput rose to 1043.49 Mcells/sec). float32 halves the bytes moved per cell. The stencil does only a few arithmetic operations per value loaded (low arithmetic intensity), so its speed is set by memory traffic rather than compute power. Halving the data therefore gives close to a 2x gain. The slightly-above-2x result is plausible because the smaller arrays fit better in CPU cache (two 1500×1500 float32 grids are about 18 MB, versus 36 MB for float64), and each SIMD instruction handles twice as many values. C. Performance is limited by $\min(\text{peak compute}, \text{memory bandwidth} \times \text{arithmetic intensity})$. A stencil has low arithmetic intensity, so it sits on the bandwidth-limited part of the Roofline curve. Once a few cores saturate the RAM bus, every extra core just waits for data. Doubling the cores doubles the available compute but not the memory bandwidth, so the speedup is far below 2x and the curve flattens. Rewrite the answers in your own words before submitting, and use the same data in your header fields (CPU model, 8 threads). If you tell me your CPU model, I can check whether the 4-core/8-thread claim in C matches your hardware.

5. Student Result Scorecard & Data Collection

Fill in your benchmark results into Table 1 below before submitting this document:

Benchmark Challenge	Active Threads	Array Size / Samples	Time (seconds)	Speedup Factor	Efficiency (%)
Challenge 1: Monte Carlo	1 Thread (Base)	120,000,000	2.5504	1.00x	100.0%
Challenge 1: Monte Carlo	2 Threads	120,000,000	1.4057	1.81	x 90.7
Challenge 1: Monte Carlo	4 Threads	120,000,000	0.8768	2.91	x 72.7
Challenge 1: Monte Carlo	Max Threads	120,000,000	0.5793	4.40	x 55.0
Challenge 2: Mandelbrot (Rows)	Max Threads	2500 x 2500	1.949	N/A	N/A
Challenge 2: Mandelbrot (Cols)	Max Threads	2500 x 2500	1.885 s	N/A	N/A
Challenge 3: Heat Stencil	Max Threads	1500 x 1500 x 300	1.397 s	N/A	N/A

Challenge 2 A. Rows took 1.949 s and columns took 1.885 s, so columns were about 3% faster. That difference is small and within normal run-to-run noise, so the two are effectively equal. The cost is dominated by the escape-time loop (up to 1000 iterations per pixel), while there is only one memory write per pixel. Memory layout therefore has little influence here. B. NumPy arrays are C-contiguous, so `img[r, c]` and `img[r, c+1]` are adjacent in memory. Row version: the inner loop over `c` writes consecutive addresses, so each 64-byte cache line (16 int32 values) is fully used. Column version: the inner loop over `r` jumps by a full row ($2500 \times 4 = 10,000$ bytes) on every write. Each write touches a different cache line and uses only a small part of it, which wastes cache and bandwidth. Writing `img[c, r]` would swap this behavior. In this lab the effect is hidden because computing a pixel costs far more than writing it. With static scheduling, each thread also works on a separate block of columns, so overlap between threads happens only at block boundaries. C. Rows near the top and bottom of the image ($|y|$ close to 1.2) lie mostly outside the Mandelbrot set. Their points escape after a few iterations, so those rows finish quickly. Rows in the center cross the interior of the set, where points run all 1000 iterations. With static scheduling, each thread gets a fixed block of rows. The thread with the top rows finishes early and idles at the barrier while the thread with the center rows is still working. Total time equals the slowest thread, so cores are wasted. Dynamic scheduling splits the rows into small chunks that threads take on demand. A thread that finishes a cheap chunk grabs another one, so all threads stay busy and finish at about the same time. D. Attach `mandelbrot_output.png`.